MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana

Department of Computer Science and Engineering

ALGORITHMIC PROBLEM-SOLVING HACKATHON (APSH 2026)

PROJECT DOCUMENTATION

Date: 03-10-2026 | Venue: 3rd Floor, A & B Blocks

Field	Details
Project Title	MRTC 3D Campus Navigation System — Shortest Walking Routes using Dijkstra's Algorithm on a Measured Campus Graph
Team Name / Team No.	
Year / Semester / Section	II Year B.Tech / ___ Sem / ___
Algorithm Paradigm Used	**Graph** — Dijkstra's Shortest Path Algorithm (binary min-heap); accessibility constraints applied as an edge filter
Programming Language / Tools	TypeScript, React, Vite, Three.js / React Three Fiber, Tailwind CSS, Vitest (testing); Google Maps Measure-distance tool (campus data); Git / GitHub
Faculty Mentor	

Team Members

S.No	Name of the Student	Roll No.	Role
1			Team Lead
2			
3			
4			

1. Problem Statement

State the problem clearly, with its inputs, outputs and constraints.

Students, staff and visitors at Malla Reddy Vishwavidyapeeth – Technical Campus (MRTC), Hyderabad, need to find the **shortest walking route** between academic blocks, hostels, the canteen, sports facilities and parking. Campus walkways have different lengths, so the route with the fewest turns is not necessarily the shortest, and public map services do not know the internal lanes (Google walking directions give 550 m for a hostel-to-hostel walk that is 234.23 m along the real campus paths). Wheelchair users additionally need routes that avoid stairs and inaccessible entrances.

Input

- A campus graph G = (V, E): **V = 18** locations (Blocks A, B, C, Canteen, Girls Hostel, NIAT Boys Hostel, 9 sports/parking facilities, 3 road junctions) and **E = 20** walkable connections, each with a measured length in metres.
- A source s and a destination t chosen by the user (or by voice), and a route preference: Shortest, Fastest, Wheelchair, Step-free or Avoid-stairs.

Output

- The ordered route s → … → t with its total distance, the distance of every segment, turn-by-turn directions and an estimated walking time — or an explicit message that no valid route exists.

Constraints

- All edge weights are positive real numbers (metres); the graph is undirected (walkways can be used both ways).
- Only real, walkable connections are edges (e.g. there is no direct Girls Hostel ↔ Boys Hostel path; the route must pass through road junctions).
- Unknown data is never invented: an unmeasured connection is disabled; unknown accessibility is reported as "unverified".
- In accessibility modes, edges with recorded stairs / steps / inaccessibility, and entrances recorded as inaccessible (Block C front), must not be used.

2. Objectives

List 2–4 specific objectives of the project.

- Model the real MRTC campus as a **weighted undirected graph** using walkway distances measured on Google Maps, with every value labelled VERIFIED / ESTIMATED / UNKNOWN.
- Implement **Dijkstra's algorithm from scratch** with a binary min-heap and route reconstruction, and verify it against an independent algorithm (Floyd–Warshall) and naive baselines.
- Support **accessibility-aware routing** by filtering edges and entrances before relaxation, without ever presenting unverified data as accessible.
- Present the computed route in an **interactive 3D campus** (Earth-to-campus transition, route animation, directions, live-GPS / simulation navigation and voice commands).

3. Proposed Approach

Explain the algorithm design paradigm chosen and why it suits this problem. Mention alternatives considered.

Paradigm: Graph. Each location (building entrance, facility gate, road junction) is a vertex; each walkway is an undirected edge whose weight is its measured length. The task — minimum total distance between two vertices with non-negative weights — is the single-source shortest-path problem, which **Dijkstra's algorithm** solves exactly and efficiently on sparse graphs.

Why it suits this problem: weights are non-negative metres, so the vertex with the smallest tentative distance can be finalised safely (any other route would pass a farther vertex and only add length). Dijkstra makes this greedy choice globally, using a min-heap, and stores a predecessor (prev) for every vertex so the actual route can be rebuilt. The search stops as soon as the destination is finalised.

Accessibility as a graph constraint: before an edge is relaxed it passes an edge filter for the selected mode (Wheelchair rejects stairs / not step-free / not accessible; Avoid-stairs rejects stairs; closed edges are always

rejected). Destinations resolve to entrances allowed in that mode, so a wheelchair route to Block C is refused explicitly because its front entrance is recorded as inaccessible. Alternative routes are produced with **Yen's k-shortest loopless paths**, which repeatedly calls Dijkstra on temporarily restricted graphs.

Alternatives considered

Alternative	Why not chosen (with evidence from our campus graph)
Breadth-First Search	Minimises the number of edges, not metres. Gives a longer route for 70 of 210 location pairs, e.g. Block A → Block C: 96.73 m (1 edge) instead of 90.05 m via Block B.
Naive greedy (always take the shortest next edge)	Local choice that cannot be undone: wrong or stuck in 173 of 210 pairs, e.g. Block A → Canteen walks into a dead end at the Football Court.
Depth-First Search / Backtracking (enumerate all paths)	Correct but exponential: a 6 × 6 grid already has 1,262,816 simple paths (≈1.56 s) versus ≈0.01 ms for Dijkstra.
Floyd–Warshall (Dynamic Programming)	All-pairs, $O(V^3)$: more than needed for one query. Used in our tests as an independent correctness oracle.
Bellman–Ford	Handles negative weights in $O(V \cdot E)$; unnecessary because walking distances are never negative.
A* search	Also optimal with an admissible heuristic and may explore fewer vertices; Dijkstra (A* with h = 0) is the required algorithm and is instant on 18 vertices.

4. Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode. Add a flowchart if needed.

Algorithm DIJKSTRA(G, s, T, mode) — T is the set of allowed destination entrances

```
1.  for each vertex v in V:  dist[v] ← ∞ ;  prev[v] ← NIL
2.  dist[s] ← 0 ;  visited ← ∅ ;  H ← empty min-heap ;  H.push(s, 0)
3.  while H is not empty:
4.      (u, d) ← H.popMin()
5.      if u ∈ visited or d > dist[u]:  continue          // stale entry (lazy deletion)
6.      visited ← visited ∪ {u}                          // dist[u] is now final
7.      if u ∈ T:  return RECONSTRUCT(prev, s, u), dist[u]
8.      for each edge (u, v, w) in Adj[u] with v ∉ visited:
9.          if EDGEFILTER(mode, edge) rejects it:  continue   // stairs, closed, ...
10.         if dist[u] + w < dist[v]:                    // RELAXATION
11.             dist[v] ← dist[u] + w ;  prev[v] ← u
12.             H.push(v, dist[v])
13. return "unreachable", ∞

RECONSTRUCT(prev, s, t):
1.  path ← [ ] ;  x ← t
2.  while x ≠ NIL:  insert x at the front of path ;  x ← prev[x]
3.  return path                                          // s → ... → t
```

Worked example on MRTC data: Block A → Canteen

Step	Vertex finalised	Relaxations	Result
1	Block A (0)	B = 36.67, C = 96.73, Basketball = 57.26, Bike Parking = 31.79	4 vertices in heap
2	Bike Parking (31.79)	Car Parking = 67.76, Volleyball = 47.09	closest first, even though off-direction
3	Block B (36.67)	C: 36.67 + 53.38 = **90.05 < 96.73** → update, prev[C] = B	relaxation improves C
…	…	…	…
—	Block C (90.05)	Canteen = 90.05 + 84.78 = 174.83	
—	Canteen (174.83)	destination reached — stop (16 of 18 vertices finalised)	Route A → B → C → Canteen = **174.83 m**

Flowchart

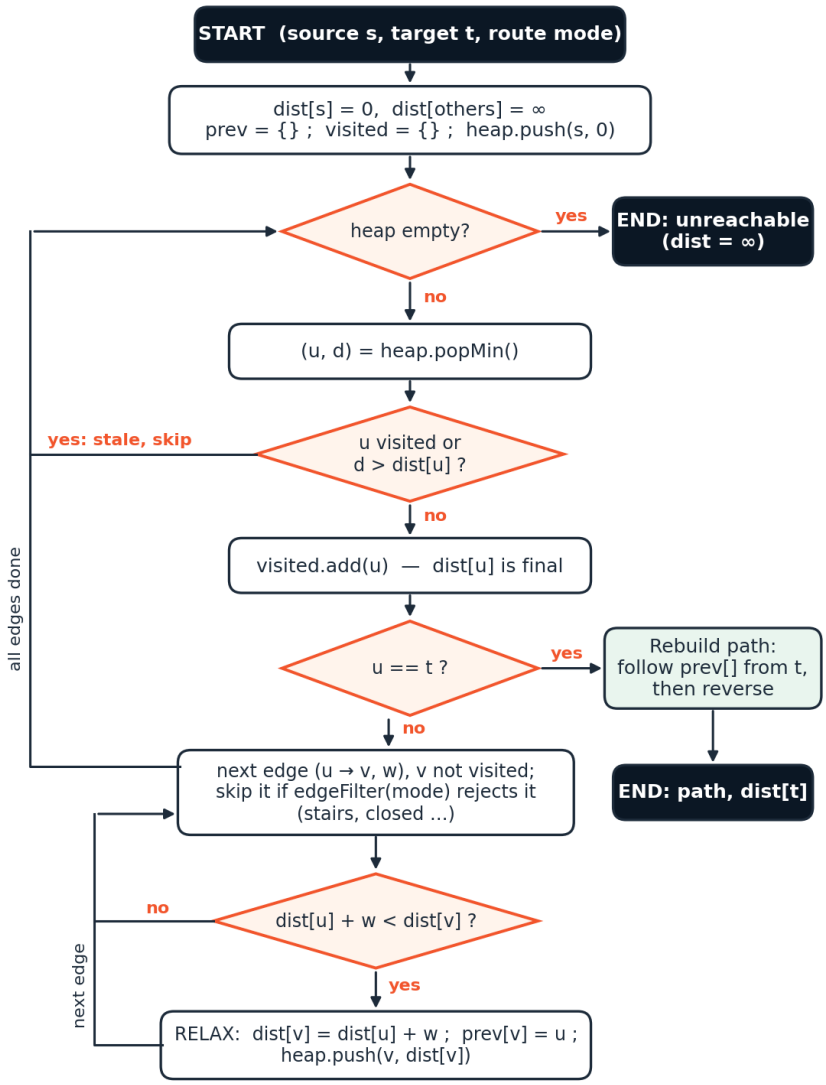


Figure 1. Flowchart of Dijkstra's algorithm with accessibility edge filter and route reconstruction.

5. Complexity Analysis

Give the time and space complexity, with justification.

Complexity	Best Case	Average Case	Worst Case
Time	O(V) — source = target: initialisation, one pop	O((V + E) log V)	O((V + E) log V) — target settled last or unreachable
Space	O(V) auxiliary (dist, prev, visited)	O(V + E)	O(V + E) — heap may hold up to 2E + 1 lazy entries

Let V = number of vertices and E = number of edges (MRTC: V = 18, E = 20). The graph is stored as an adjacency list; the priority queue is a binary min-heap with lazy deletion.

- **Heap operations:** each successful relaxation pushes one entry, so there are at most 2E + 1 pushes (each undirected edge appears twice in the adjacency list) and at most as many pops. Each push/pop costs O(log(heap size)) = O(log E) = O(log V), because $E \leq V^2$.
- **Edge scanning:** each vertex's adjacency list is scanned once, when it is finalised — O(V + E) in total.
- **Total time:** O((V + E) log V). **Space:** O(V + E) for the graph, plus O(V) for dist / prev / visited and up to O(E) heap entries.

- **Best case:** source equals destination — initialisation O(V) and a single pop. **Worst case:** destination finalised last or unreachable, so every vertex and edge is processed.
- **Implementation note:** when the step-by-step trace for the visual Algorithm panel is enabled, each recorded step copies the distance table (O(V) per step). Route alternatives (Yen's algorithm, K routes) cost O(K · V · (V + E) log V).

6. Implementation

Describe the main modules or functions, the data structures used and the tools or platform.

The system is a browser application written in **TypeScript** with **React** (user interface), **Three.js / React Three Fiber** (3D globe and campus) and **Vite** (build tool). The algorithmic core — Graph, MinHeap, Dijkstra, Yen's alternatives, accessibility filter and map matching — is written from scratch without any graph library. Source: github.com/vidyaasagaar/campus-navigation.

Main modules

Module / file	Responsibility	Data structures
data/campusMeasurements.json	Locations, measured walkway distances, walkway polylines, accessibility and verification metadata	JSON records
graph/Graph.ts	Undirected weighted graph; rejects unknown nodes, non-positive and duplicate edges	Map (hash map) + adjacency list
algorithms/MinHeap.ts	Priority queue with sift-up / sift-down	Binary min-heap (array)
algorithms/dijkstra.ts	Shortest path, early exit, multi-target entrances, edge filter, step trace	dist Map, prev Map, visited Set, heap
algorithms/alternativeRoutes.ts	Route 2 / Route 3 via Yen's k-shortest loopless paths	lists, sets
algorithms/accessibility.ts	Route modes, edge filter, accessible-entrance selection, warnings	predicate functions
algorithms/baselines.ts	BFS and naive greedy, for comparison only	queue, set
navigation/session/*	Live GPS filtering, map matching (point-to-segment projection), rerouting, simulation, arrival detection	polyline arrays
campus/, globe/	3D campus, sports surfaces, labels; Earth globe and camera flight	Three.js meshes
voice/*	Speech-to-intent parser, read-aloud text, text-to-speech	alias tables

Data flow

campusMeasurements.json → buildCampusGraph() → Graph → edgeFilter(mode) → dijkstra() → path + distance → (a) 3D route renderer, (b) directions and ETA, (c) voice read-aloud, (d) live / simulated navigation session. The 3D layer only displays the algorithm's output; changing a distance in the data file changes the route without code changes.

Campus data collection

Every walkway was traced on Google Maps satellite imagery with the **Measure distance** tool (along the path, not straight-line) and cross-checked with Google walking directions where available. Values are labelled ESTIMATED (map-measured) until verified on the ground. Building identities and facility layout (parking, football ground, box cricket, courts, entrance accessibility) were confirmed by the team.

7. Results and Test Cases

Give sample inputs and outputs and attach screenshots. Compare performance with a naive approach, if applicable.

Sample inputs and outputs (real campus graph)

#	Input (source → destination, mode)	Output (route)	Distance
1	Block A → Canteen, Shortest	A → B → C → Canteen	174.83 m
2	Block A → Block C, Shortest	A → B → C (beats direct A → C = 96.73 m)	90.05 m
3	Girls Hostel → Boys Hostel, Shortest	via hostel-road junction, lane junctions, Badminton Court, Box Cricket	234.23 m

#	Input (source → destination, mode)	Output (route)	Distance
4	Block A → Box Cricket, Shortest	A → Basketball → … → Badminton → Box Cricket	228.40 m
5	Canteen → Block B, Wheelchair	Canteen → C → B; ✓ accessible entrance at B (user-verified); 2 segments unverified	138.16 m
6	Any → Block C, Wheelchair	No route: Block C front recorded as not wheelchair accessible; no other entrance known	—
7	Test graph: A–B 100 (stairs), A–C 60, C–B 80; Wheelchair	A → C → B (stairs edge rejected)	140 m (normal: 100 m)
8	Test graph: A–B 10, A–C 3, C–B 2	A → C → B	5
9	Unreachable destination / invalid node	empty path, ∞ / explicit error	—

Comparison with naive approaches

Approach	Correct?	Evidence
Dijkstra (ours)	Always optimal	Matches Floyd–Warshall on 30 random graphs and brute force on all 210 campus pairs
BFS (fewest edges)	No	Longer route for 70 / 210 campus pairs
Naive greedy	No	Wrong or stuck for 173 / 210 pairs (136 dead ends, 37 longer)
Brute force (all simple paths)	Yes, but exponential	Campus: ≤ 8 paths per pair, ≈0.004 ms (same as Dijkstra). 6 × 6 grid (V = 36, E = 60): 1,262,816 paths, ≈1557 ms vs ≈0.013 ms for Dijkstra

Timings were measured on the development laptop (Node.js) and are indicative only. On the small campus graph both methods are instant; the grid experiment shows how brute force explodes as the network grows (for example when indoor rooms and corridors are added), while Dijkstra stays at $O((V + E) \log V)$.

Automated testing

71 automated tests in 7 files (Vitest) pass, covering: shortest paths and route reconstruction; unreachable, same-node, tie, invalid-node and negative-weight cases; 30 random graphs against Floyd–Warshall; the six accessibility scenarios (normal, wheelchair, lift unavailable, ramp closed, no accessible route, accessible entrance); indoor graph rules; map matching, GPS outlier rejection and rerouting; and voice-command parsing including ambiguity handling.

Screenshots

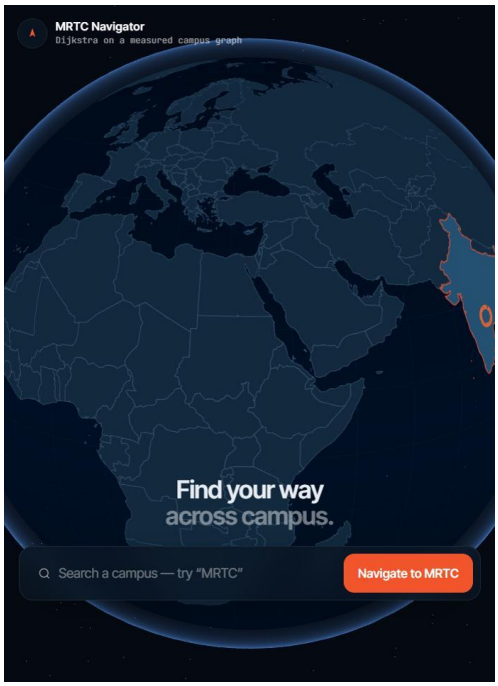


Figure 2. Globe view and campus search (start of the Earth-to-campus flight).

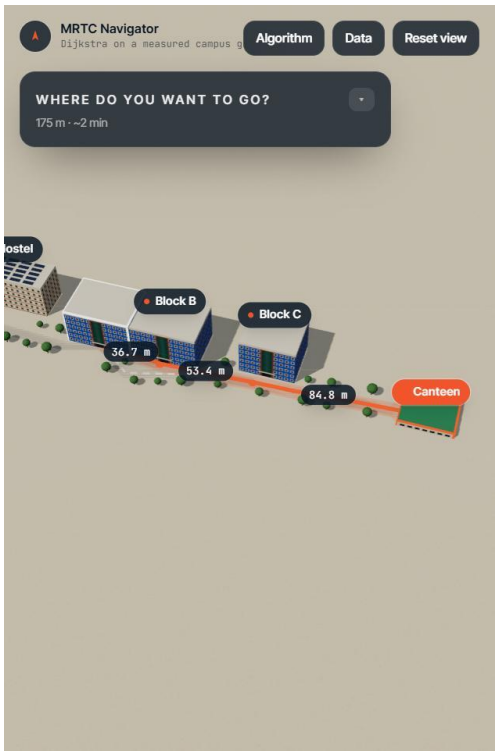


Figure 3. Computed route Block A → Canteen with segment distances 36.7 m, 53.4 m and 84.8 m.

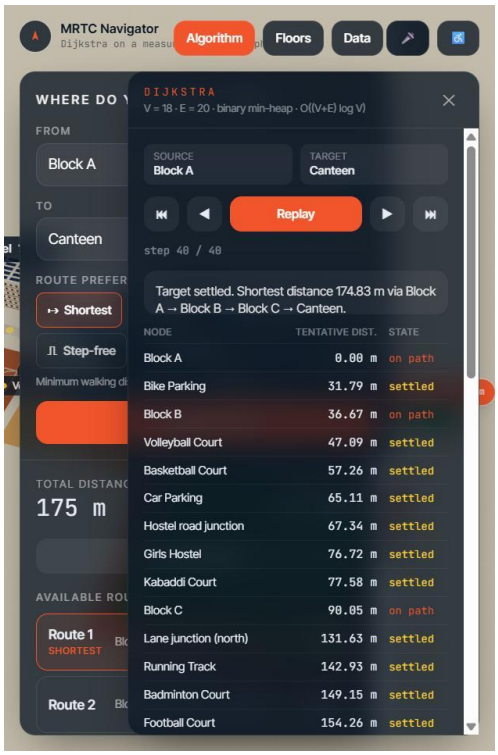


Figure 4. Algorithm panel: Dijkstra trace for Block A → Canteen (final distance 174.83 m).

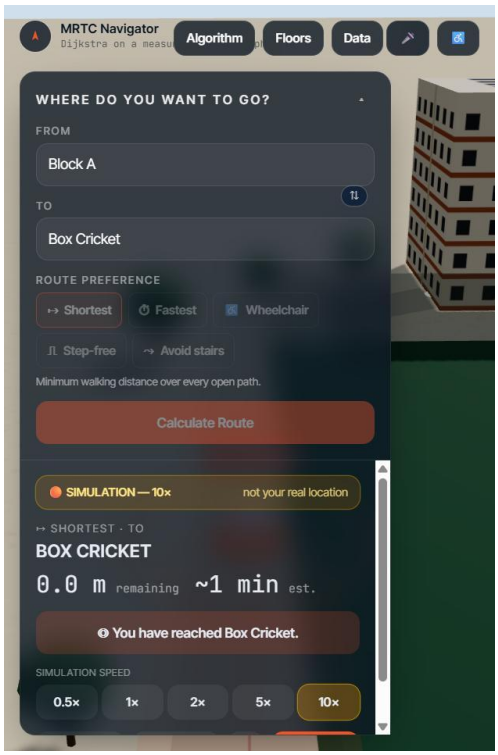


Figure 5. Simulation mode (labelled, 10×): arrival at Box Cricket detected from position.

8. Conclusion and Future Scope

Summarise the outcome and the improvements possible.

We built a campus navigation system in which every route is produced by **Dijkstra's shortest-path algorithm** running on a weighted graph of the real MRTC campus. The implementation is written from scratch (graph, binary min-heap, Dijkstra, Yen's alternatives), is verified by 71 automated tests including an independent Floyd–Warshall cross-check, and demonstrably outperforms naive BFS, greedy and brute-force approaches.

Accessibility is part of the algorithm (edge filtering and entrance selection), and the system is explicit about what is measured, estimated, unknown or simulated.

Future scope

- Walk each edge with a measuring wheel / GPS logger and mark distances VERIFIED; survey kerbs, ramps, slopes and entrances for accessibility.
- Activate indoor navigation: room numbers, corridors, staircases and lifts as graph nodes (the model for 2 west + 3 east classrooms and HOD/Staff rooms is already implemented).
- Indoor positioning with BLE beacons or QR checkpoints; real-time closures (ramp or lift out of service) entered by administrators.
- A* with a straight-line heuristic and zone partitioning (divide & conquer) if the graph grows to thousands of vertices.

9. References

List textbooks, research papers and online resources (IEEE format).

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[3] J. Y. Yen, "Finding the k shortest loopless paths in a network," *Management Science*, vol. 17, no. 11, pp. 712–716, 1971.

[4] R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, vol. 5, no. 6, p. 345, 1962.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[6] W3C, "Geolocation API," W3C Recommendation. [Online]. Available: https://www.w3.org/TR/geolocation/ (accessed Oct. 3, 2026).

[7] Mozilla, "Web Speech API," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API (accessed Oct. 3, 2026).

[8] three.js authors, "three.js documentation." [Online]. Available: https://threejs.org/docs/ (accessed Oct. 3, 2026).

[9] Natural Earth, "Admin 0 – Countries" (public domain), via the world-atlas package. [Online]. Available: https://www.naturalearthdata.com/ (accessed Oct. 3, 2026).

[10] Google, "Measure distance between points," Google Maps Help. [Online]. Available: https://support.google.com/maps/answer/1628031 (accessed Oct. 3, 2026).

[11] Project repository, "MRTC 3D Campus Navigation." [Online]. Available: https://github.com/vidyaasagaar/campus-navigation